

# Stored Procedure

```
DELIMITER //
```

```
create procedure GetRoomsWithMetadata(IN currentDate varchar(10), IN
currentTime varchar(10))
BEGIN
    SELECT
        r.RoomNumber,
        r.BuildingId,
        r.Capacity,
        r.Type,
        b.BuildingName,
        (
            NOT EXISTS (
                SELECT 1
                FROM UserReservations ur
                WHERE ur.BuildingId = r.BuildingId
                    AND ur.RoomNumber = r.RoomNumber
                    AND STR_TO_DATE(ur.Date, '%Y-%m-%d') =
STR_TO_DATE(currentDate, '%Y-%m-%d')
                    AND STR_TO_DATE(currentTime, '%H:%i') BETWEEN
STR_TO_DATE(ur.StartTime, '%H:%i') AND STR_TO_DATE(ur.EndTime, '%H:%i')
            )
            AND NOT EXISTS (
                SELECT 1
                FROM HardReservations hr
                WHERE hr.BuildingId = r.BuildingId
                    AND hr.RoomNumber = r.RoomNumber
                    AND STR_TO_DATE(currentDate, '%Y-%m-%d') BETWEEN
STR_TO_DATE(hr.StartDate, '%Y-%m-%d') AND STR_TO_DATE(hr.EndDate, '%Y-%m-%d')
                    AND STR_TO_DATE(currentTime, '%H:%i') BETWEEN
STR_TO_DATE(hr.StartTime, '%H:%i') AND STR_TO_DATE(hr.EndTime, '%H:%i')
            )
        ) AS `Available`
    FROM Rooms r
    JOIN Buildings b ON r.BuildingId = b.BuildingId
    WHERE b.Latitude IS NOT NULL AND b.Longitude IS NOT NULL;
END //
```

```
DELIMITER ;
```

```
DELIMITER //
```

```
create procedure GetNearestAvailableBuildings(
    IN userLat decimal(10, 8), IN userLong decimal(11, 8),
    IN currentDate varchar(10), IN currentTime varchar(10),
```

```

    IN maxResults int
)
BEGIN
    IF userLat < -90 OR userLat > 90 THEN
        SELECT 'Invalid latitude: must be between -90 and 90' AS ErrorMessage;
    ELSEIF userLong < -180 OR userLong > 180 THEN
        SELECT 'Invalid longitude: must be between -180 and 180' AS
ErrorMessage;
    ELSEIF maxResults <= 0 THEN
        SELECT 'Invalid maxResults: must be greater than 0' AS ErrorMessage;
    ELSE
        SELECT
            BuildingId,
            BuildingName,
            Latitude,
            Longitude,
            6371 * 2 * ASIN(SQRT(
                POWER(SIN(RADIANS((Latitude - userLat) / 2)), 2) +
                COS(RADIANS(userLat)) * COS(RADIANS(Latitude)) *
                POWER(SIN(RADIANS((Longitude - userLong) / 2)), 2)
            )) AS DistanceKm,
            COUNT(DISTINCT RoomNumber) AS FreeRoomCount
        FROM (
            SELECT
                r.RoomNumber,
                r.BuildingId,
                b.BuildingName,
                b.Latitude,
                b.Longitude
            FROM Rooms r
            JOIN Buildings b ON r.BuildingId = b.BuildingId
            WHERE b.Latitude IS NOT NULL AND b.Longitude IS NOT NULL
                AND NOT EXISTS (
                    SELECT 1
                    FROM UserReservations ur
                    WHERE ur.BuildingId = r.BuildingId
                        AND ur.RoomNumber = r.RoomNumber
                        AND STR_TO_DATE(ur.Date, '%Y-%m-%d') =
STR_TO_DATE(currentDate, '%Y-%m-%d')
                        AND STR_TO_DATE(currentTime, '%H:%i') BETWEEN
STR_TO_DATE(ur.StartTime, '%H:%i') AND STR_TO_DATE(ur.EndTime, '%H:%i')
                )
                AND NOT EXISTS (
                    SELECT 1
                    FROM HardReservations hr
                    WHERE hr.BuildingId = r.BuildingId
                        AND hr.RoomNumber = r.RoomNumber
                        AND STR_TO_DATE(currentDate, '%Y-%m-%d') BETWEEN
STR_TO_DATE(hr.StartDate, '%Y-%m-%d') AND STR_TO_DATE(hr.EndDate, '%Y-%m-%d')

```

```

                AND STR_TO_DATE(currentTime, '%H:%i') BETWEEN
STR_TO_DATE(hr.StartTime, '%H:%i') AND STR_TO_DATE(hr.EndTime, '%H:%i')
            )
        ) AS AvailableRooms
    GROUP BY BuildingId, BuildingName, Latitude, Longitude
    HAVING FreeRoomCount > 0
    ORDER BY DistanceKm ASC
    LIMIT maxResults;
END IF;
END //

DELIMITER ;

```

## Trigger

```

DELIMITER //

create trigger insert_room_if_missing
    before insert
    on HardReservations
    for each row
begin
    IF NOT EXISTS (
        SELECT 1 FROM Rooms
        WHERE RoomNumber = NEW.RoomNumber AND BuildingId = NEW.BuildingId
    ) THEN
        INSERT INTO Rooms (RoomNumber, BuildingId)
        VALUES (NEW.RoomNumber, NEW.BuildingId);
    END IF;
END //

DELIMITER ;

```

## Transaction

```

data = request.json
building_id = data.get('BuildingId')
room_number = data.get('RoomNumber')
date = data.get('Date')
start_time = data.get('StartTime')
end_time = data.get('EndTime')

connection = get_db_connection()
cursor = connection.cursor()
cursor.execute("SET TRANSACTION ISOLATION LEVEL REPEATABLE READ")
connection.start_transaction()

```

```

try:
    # 1. Check weekly reservation limit
    cursor.execute("""
        SELECT BuildingName, COUNT(*)
        FROM UserReservations
        NATURAL JOIN Buildings
        WHERE UID = %s AND YEARWEEK(Date, 1) = YEARWEEK(%s, 1)
        GROUP BY BuildingName
    """, (uid, date))

    existing_reservations = cursor.fetchall()
    if existing_reservations:
        data = {}
        total_reservations = 0
        for reservation in existing_reservations:
            data[reservation[0]] = reservation[1]
            total_reservations += reservation[1]
        if total_reservations > 5:
            return jsonify({'error': f'You already have 5 reservations in a
week: {data}'}), 400

    # 2. Check for time conflicts
    cursor.execute("""
        SELECT ReservationId AS id
        FROM UserReservations
        WHERE BuildingId = %s AND RoomNumber = %s
        AND EndTime > %s AND StartTime < %s AND Date = %s
        UNION ALL
        SELECT EventId AS id
        FROM HardReservations
        WHERE BuildingId = %s AND RoomNumber = %s
        AND EndTime > %s AND StartTime < %s
        AND Repeats LIKE %s
    """, (
        building_id, room_number, start_time, end_time, date,
        building_id, room_number, start_time, end_time,
        f"%{get_weekday_letter(date)}%"
    ))

    conflicts = cursor.fetchall()
    num_conflicts = len(conflicts)
    if num_conflicts:
        connection.rollback()
        return jsonify({'error': f'Conflicted with {num_conflicts} reservations.
Try another slot.'}), 409

    # 3. Actually insert the new reservation
    cursor.execute("""

```

```

        INSERT INTO UserReservations (UID, RoomNumber, BuildingId, Date,
StartTime, EndTime)
        VALUES (%s, %s, %s, %s, %s, %s)
        """%(uid, room_number, building_id, date, start_time, end_time))

    # 4. If all successful, commit
    connection.commit()

    return jsonify({'message': 'User Reservation added successfully'}), 201

except mysql.connector.Error as err:
    connection.rollback()
    return jsonify({'error': str(err)}), 500
finally:
    cursor.close()
    connection.close()

```

## Constraints

In the below create table statement, we define attribute level “NOT NULL” constraints on all attributes, and we additionally define primary and foreign keys.

```

CREATE TABLE UserReservations (
    ReservationId INT PRIMARY KEY AUTO_INCREMENT,
    Date VARCHAR(10) NOT NULL,
    StartTime VARCHAR(10) NOT NULL,
    EndTime VARCHAR(10) NOT NULL,
    UID INT NOT NULL,
    RoomNumber VARCHAR(10) NOT NULL,
    BuildingId VARCHAR(7) NOT NULL,
    FOREIGN KEY (UID) REFERENCES Users(UID) ON DELETE CASCADE,
    FOREIGN KEY (RoomNumber, BuildingId) REFERENCES Rooms(RoomNumber,
BuildingId) ON DELETE CASCADE
);

```